2.1 根因分析

横向树图 · 待确认疑因

Lock / RCU 告警只给出入口：还要查清谁在等、什么没前进，以及平台当时是否正常

定位时间长	领域与工作难点	导致反复排查的具体疑因
Lock / RCU 定位周期长	等待关系难还原 内核机制	**F1 跨 CPU 查等锁链，靠人工逐个还原** 多核同卡慢路径；qspinlock 需结合锁地址、其他 CPU 栈与 MCS 节点追查
		F2 RCU 卡在哪一步，需要反复比对 同一类 stall 可能来自缺 QS、GP 线程饥饿或 timer 唤醒失败
	软硬件原因难分 ESL / FPGA	**F3 低频、计时异常与真停滞混在一起** FPGA 主频/访存慢；ESL 计时模式各异，要结合进度、计时器和 IRQ 判断
		F4 互联或中断异常，软硬证据对不上 互联响应、IRQ 投递可能异常；串口、硬件 trace、仿真事件要对到同一现场
	补采与复现反复 日常排查	**F5 拿到现场后，材料仍不够用** vmcore、同版符号、全 CPU 栈与固件/镜像未成套，补采后才能继续
		F6 复现一轮很久，却没拿到要验证的证据 低性能平台启动和触发慢；新增 printk / trace 又可能改变负载或时序

改进重点：先利用已有现场减少盲查和补采，把昂贵的下一轮复现用于验证明确的假设。

源码与公开资料用于说明机制；六项工作疑因的发生情况、影响时长及主次，须由案例记录确认。

2.2 主疑因筛选

调查表 · 按耗时排序

逐项核查补材料、人工分析和重复复现花了多久，再按可避免耗时排先后

先把时间拆开：取证与交接等待 / 人工分析工时 / 平台运行与复现等待

疑因	从原问题单核查什么	Agent 所需输入与可做的工作
F1 跨 CPU 查锁	查了多少 CPU / 任务、跑了几轮命令？ 哪些时间花在重复翻栈、查锁对象？	vmcore + 同版符号 → crash / drgn 批量查栈和锁； 可先封装，保留命令、地址和原始输出。
F2 判断 RCU 分支	多少次告警被反复比对？ 是否把报告 CPU 当成停滞 CPU 追查？	多次 stall + vmcore / RCU trace → 按 GP、CPU 比对 QS、GP 线程和 timer 线索。
F3 核对平台时序	换频率、调超时、重新跑了几轮？ 何时才查清当时的频率与计时口径？	CPU/内存/互联频率 + 计时模式 + IRQ/进度记录 → 区分执行慢、时钟/中断异常与证据不足。
F4 对齐软硬证据	等平台日志、补寄存器和来回沟通多久？ 双方记录能否定位同一次异常？	串口 + 互联/仿真 trace + 寄存器快照 → 按共同事件对齐，列出需芯片侧确认的问题。
F5 补齐现场材料	是否重新索取符号、镜像或 dump？ 关键对象不可读是否造成再次取证？	vmcore + vmlinux/模块符号 + 镜像清单 → 校验版本、检查关键对象可读性，列缺项。
F6 重复复现	每轮想排除什么、实际新增了什么证据？ 哪些运行因观察点不够而重做？	旧脚本 + 当前假设 + 平台限制 → 选观察点、写预期现象与停止条件。

工具可从离线 vmcore 分析和材料检查起步；要因仍按实际耗时排序，全部六项逐条核查。

筛选依据：案例事实与耗时。工具实现顺序不等同于要因排名；具体字段与版本要求见备注及输入清单。

2.3 根因验证

旧案例核查 · 待实测

直接使用留存的 dump、日志、脚本和交接记录，查明哪些环节确实造成了延迟

首轮建议：锁等待、RCU stall、ESL / FPGA 问题各取 1 个已结单案例；未覆盖的疑因保留待确认。

疑因	用现有材料怎么核实	确认依据与记录
F1 跨 CPU 查锁	回看旧 dump 的命令记录；对照结单依据， 标出找等锁者、锁地址和相关 CPU 的步骤。	确认重复操作确实拖慢分析； 记录人工工时及错误 / 漏查项。
F2 判断 RCU 分支	按同一 GP、CPU 比较连续告警或 trace； 回查何时才分清缺 QS、GP 饥饿或 timer 问题。	记录分支判断的返工次数； 补齐哪条证据后，分析方向才改变。
F3 核对平台时序	用已有运行记录核对镜像、负载、频率、 计时模式、IRQ 与进度；比较当时的判断。	确认平台差异是否导致误判 / 返工； 单凭调大超时后不报警，不能确认。
F4 对齐软硬证据	按共同事件对齐串口与现有平台记录； 请芯片侧核实异常事务 / IRQ 是否成立。	记录补证与交接等待； 无对应硬件证据，保留未确认。
F5 补齐现场材料	实际加载旧 dump，核对符号和关键对象； 回查补齐前后，哪一步检查才得以继续。	留存加载结果与补材料记录； 确认是否造成额外等待或重新取证。
F6 重复复现	比较每轮脚本、观察点和得到的日志； 找出没有回答当前疑问、因而重跑的轮次。	记录无效轮次及运行时间； 找出后来增加的有效观察点。

另做 vmcore 小试：隐藏结单答案，人工与 Agent 按同一清单检查；核对结果，并计入加载、分析、复核与补查工时。

根因确认看证据及耗时贡献；Agent 是否更快另做小试，以核对后的实测结果为准。

2.4 拟定对策

先把工程师每天重复做的检查交给工具，再让 Agent 根据结果追问和查代码

先做两件事：材料能不能用 / 用旧 vmcore 把 CPU、锁与 RCU 状态一次查齐

对策 What	目标 Why	措施 How	负责人 Who	时间 When	地点 Where
F5 现场包检查	必查项逐项有结果 缺项一次列清	vmcore + 同版 vmlinux / 模块符号 + 镜像清单 → 核对版本和关键对象可读性，输出缺项及影响。	内核分析 工具开发	第 1 周	离线分析机
F1 / F2 vmcore 批量分析	旧案例关键项无错漏 初查 + 复核工时 逐例对比	按内核版本封装 crash / drgn：全 CPU 栈、mutex owner、qspinlock 队列、RCU 状态；保留命令、地址和输出。	内核分析 工具开发	第 2 周	离线分析机
F3 / F4 平台联查	列清可疑 CPU / 地址及待核实的事务 / IRQ 每项注明证据出处	频率 / 计时模式 + IRQ / 进度记录 + 串口 / 硬件 trace → 对齐同一事件，生成给芯片侧的具体核查项。	内核分析 芯片 / 平台	第 3 周	原平台记录 离线分析机
F6 定向复现	每轮验证 1 个假设 有预期现象 与停止条件	现有脚本 + 当前假设 → 查代码选最小观察点，限定采集范围和时长；测试人员审查后在原平台执行。	内核分析 测试 / 平台	第 3–4 周	ESL / FPGA 或原故障平台

验收目标：常规问题平均定位 1–2 个工作日

受理到根因证据确认；分析工时、平台运行与协同等待分别记录，ESL / FPGA 按同一范围分层比较。

角色、排期与指标为拟定值；工具输入清单随材料归档。实际提效以旧案例小试和后续问题单统计为准。